

🔗 Comandos de "Sobrevivência" e Produtividade no Git

Comando	Funcionalidade	Exemplo Prático
<code>git log</code> <code>--</code> <code>oneline</code> <code>--graph</code>	Mostra o histórico de commits de forma visual, resumida em uma linha e com ramificações coloridas.	<code>git log --oneline --graph</code>
<code>git diff</code>	Mostra exatamente quais linhas foram apagadas (em vermelho) ou adicionadas (em verde) antes de você fazer o commit.	<code>git diff index.html</code>
<code>git commit -amend -m</code>	Errou a mensagem do último commit ou esqueceu de adicionar um arquivo? Esse comando "refaz" o último commit em vez de criar um novo.	<code>git commit --amend -m "Corrige texto da tag"</code>
<code>git stash</code>	A "gaveta". Guarda alterações não finalizadas temporariamente para você mudar de branch ou atualizar o projeto, sem precisar fazer um commit incompleto.	<code>git stash</code> (guarda)

`git stash pop` (recupera) |

| `git restore` | Desfaz todas as alterações que você fez em um arquivo e o devolve exatamente como estava no último commit salvo. | `git restore script.js` |

| `git switch -c` | Comando moderno para criar uma nova *branch* (ramificação) e já pular para dentro dela automaticamente. | `git switch -c feature-nova` |

| `git reset --soft HEAD~1` | Desfaz o último commit, mas **mantém** as suas alterações salvas na máquina. Ótimo se você comitou no lugar errado. | `git reset --soft HEAD~1` |

| `git reset --hard HEAD~1` | ⚠️ **Cuidado:** Apaga o último commit e **destrói** todas as alterações feitas nele. Volta no tempo de forma agressiva. | `git reset --hard HEAD~1` |

| `git blame` | O famoso "dedo duro". Mostra quem foi a pessoa que alterou cada linha de um arquivo e quando isso aconteceu. | `git blame index.html` |

| `git clean -fd` | Apaga todos os arquivos e pastas da sua máquina que não estão sendo rastreados pelo Git (arquivos lixo gerados sem querer). | `git clean -fd` |

💡 Dica extra de estudo:

Você pode brincar com o **Git Aliases**, que permite criar apelidos curtos para comandos longos. Por exemplo, se você digitar isso no terminal:

```
git config --global alias.arvore "log --oneline --graph --all"
```

A partir desse momento, sempre que você (ou seus alunos) digitarem `git arvore`, o Git vai executar aquele comando longo do histórico visual! É uma ótima forma de personalizar o fluxo de trabalho.

Aqui estão alguns dos comandos de manipulação e uso mais práticos para o dia a dia. Eles cobrem desde **visualizar o status dos arquivos**, **desfazer bobagens antes de salvar**, **gerenciar pastas/arquivos** e até **esconder coisas que você não quer enviar para o GitHub**.

Comandos de Manipulação de Uso no Git

Comando	Funcionalidade	Exemplo Prático
<code>git status -s</code>	Exibe o status dos arquivos de forma ultra-compacta (M = Modificado, ?? = Novo, A = Adicionado).	<code>git status -s</code>
<code>git rm --cached</code>	Remove um arquivo do rastreamento do Git sem apagá-lo da sua máquina . Ótimo para quando você enviou uma pasta sem querer.	<code>git rm --cached node_modules/</code>
<code>git mv</code>	Renomeia ou move um arquivo/pasta no projeto. O Git já entende a mudança automaticamente sem perder o histórico do arquivo.	<code>git mv teste.html index.html</code>
<code>git checkout --</code>	"Desfaz" as edições do arquivo antes do <code>git add</code> , voltando ao estado exato do último commit.	<code>git checkout -- estilo.css</code>
<code>git restore --staged</code>	"Tira da fila" do commit. Se você deu <code>git add .</code> sem querer, esse comando faz o arquivo voltar ao estado de rascunho.	<code>git restore -- staged arquivo.js</code>
<code>git log -p</code>	Mostra o histórico de commits exibindo as linhas exatas que foram alteradas dentro de cada um deles.	<code>git log -p -2</code> (Mostra os últimos 2)
<code>git log --author</code>	Filtra o histórico para mostrar apenas as alterações feitas por um usuário específico.	<code>git log -- author="Nome"</code>
<code>git branch -m</code>	Renomeia a branch atual onde você está trabalhando (por exemplo, de <code>master</code> para <code>main</code>).	<code>git branch -m main</code>

Manipulações Essenciais para Praticar no Projetos

1. Criando o `.gitignore` (Como ignorar arquivos indesejados)

Quando criamos arquivos temporários, senhas, senhas de banco ou pastas pesadas (como `node_modules` ou `.vscode`), **não devemos enviar para o GitHub**.

1. Na raiz do projeto, crie um arquivo chamado **.gitignore** (começa com ponto mesmo).
2. Escreva dentro dele os nomes dos arquivos ou pastas que o Git deve ignorar:

```
# Ignorar configurações do editor
.vscode/

# Ignorar pastas pesadas
node_modules/

# Ignorar arquivos temporários de sistema
.DS_Store
Thumbs.db
```

(O Git simplesmente passará a fingir que esses arquivos não existem).

2. Usando o **git stash** para trocar de foco rápido

Imagina que você está no meio da resolução de um exercício da Lista 4, mas precisa trocar de pasta ou baixar algo do professor e não quer fazer um commit incompleto.

1. **Guarde suas mudanças atuais numa "gaveta":**

```
git stash
```

2. *(Sua máquina volta ao estado limpo do último commit, como se você não tivesse mexido em nada).*

3. **Puxe a atualização do professor ou faça outra coisa:**

```
git pull upstream main
```

4. **Pegue suas alterações de volta da "gaveta":**

```
git stash pop
```

Com certeza! Tem algumas "cartas na manga" tanto do Git quanto do próprio GitHub que parecem mágica. Quando se lida com alunos em laboratório, sempre acontece de tudo um pouco, então essas ferramentas são verdadeiros salva-vidas para ajudar a turma a recuperar projetos perdidos ou para otimizar o seu tempo na hora da correção.

Aqui estão alguns recursos avançados e truques sensacionais:

1. O "Botão de Pânico" Definitivo (`git reflog`)

Se um aluno chegar em desespero dizendo: "*Professor, digitei um comando de reset errado e apaguei todos os meus commits!*", o `git log` não vai ajudar, porque ele só mostra o que sobreviveu. Mas o Git registra **absolutamente todos** os passos ocultamente.

- **Comando:** `git reflog`
- **Como funciona:** Ele lista um histórico de todas as ações feitas no Git (até as exclusões e resets). Cada ação tem um código curtinho (ex: `a1b2c3d`).
- **Como salvar a pátria:** Basta achar o código de antes da tragédia e digitar `git reset --hard a1b2c3d`. O projeto volta à vida exatamente como estava naquele momento.

2. O Caçador de Bugs (`git bisect`)

Sabe quando um layout em CSS estava funcionando perfeitamente ontem, mas hoje tudo quebrou e você não faz ideia de qual alteração causou o problema?

- **Como funciona:** O `bisect` faz uma "busca binária" no histórico.
- **Como usar:**
 1. Você digita `git bisect start`.
 2. Digita `git bisect bad` (para dizer que o código atual está quebrado).
 3. Digita `git bisect good <codigo-do-commit-antigo>` (aponta um commit de dias atrás onde tudo funcionava).
 4. O Git vai pular pro meio do caminho e te perguntar: "Aqui está funcionando?". Você testa a página e responde `git bisect good` ou `git bisect bad`. Ele vai dividindo o histórico pela metade até cuspir na tela exatamente qual foi a linha e o commit que introduziu o bug.

3. O "Rouba-Cena" (`git cherry-pick`)

Imagine que você tem várias branches e percebe que fez uma alteração excelente (tipo uma correção no arquivo de Javascript) na branch errada, ou quer puxar apenas um exercício específico da Lista 3 sem trazer a lista inteira junto.

- **Comando:** `git cherry-pick <codigo-do-commit>`
- **Como funciona:** Ele vai em outra branch, copia **apenas aquele commit específico** que você pediu, e cola na branch onde você está agora.

✳ Dica de Ouro fora do Terminal: O "Ponto Mágico" no GitHub

Essa é incrível para agilizar o seu lado na hora de corrigir os códigos da turma sem precisar fazer o *clone* do repositório de cada um deles para o seu computador.

1. Abra qualquer repositório de um aluno no GitHub direto no seu navegador.
2. Simplesmente pressione a tecla **.** (**Ponto**) no seu teclado.

3. O GitHub vai abrir instantaneamente uma versão completa do **VS Code direto no navegador**, com todos os arquivos do projeto abertos para você ler, analisar a estrutura das pastas e conferir o código, com a mesma interface que usamos no desktop.

No ambiente do laboratório, com os alunos intercalando entre as máquinas da escola e os computadores de casa, alguns erros são clássicos e acontecem todo semestre.

Aqui estão as soluções para os 5 problemas mais comuns que a turma vai enfrentar e como você pode ajudá-los a "desfazer a bagunça" de forma simples:

1. "Professor, o terminal abriu uma tela estranha e travou!" (A Armadilha do Vim)

O que aconteceu: O aluno fez um `git pull` e o Git precisou juntar (merge) as atualizações, abrindo o editor de texto padrão do terminal (o Vim) para ele confirmar a mensagem. O aluno entra em pânico e começa a apertar botões aleatórios.

Como corrigir:

Fale para o aluno tirar as mãos do teclado e seguir estes passos exatos:

1. Aperte a tecla `ESC` (para garantir que ele saiu do modo de digitação).
2. Digite `:wq` (os dois pontos vão aparecer no rodapé do terminal).
3. Aperte `Enter`.

Pronto, o terminal salva o merge e volta ao normal.

2. "Fui dar push e o GitHub bloqueou com um erro vermelho de 'Rejected'"

O que aconteceu: O famoso erro de sincronia. O aluno fez a Lista 3 em casa no final de semana e enviou para o GitHub. Chegou na aula, sentou no computador, esqueceu de dar `pull` e começou a fazer a Lista 4. Quando tenta dar o push, o GitHub recusa porque a máquina do laboratório está "atrasada" em relação à nuvem.

Como corrigir:

Ele precisa puxar o que está na nuvem e mesclar com o que ele acabou de fazer.

```
# 1. Puxar as alterações de casa
git pull origin main

# 2. Se não der conflito, o Git junta tudo. Agora é só enviar:
git push origin main
```

3. "Mande a pasta errada pro commit!" (Ex: mandou a pesada `node_modules`)

O que aconteceu: O aluno deu um `git add .` sem ter configurado o arquivo `.gitignore`, e acabou adicionando o laboratório inteiro ou pastas pesadas que não deveriam subir.

Como corrigir:

Em vez de apagar a pasta do computador, ele deve usar um comando para mandar o Git esquecer aquele arquivo/pasta apenas no rastreamento:

```
# Para remover o rastreamento de uma pasta inteira sem apagá-la do PC:
git rm --cached -r nome-da-pasta/

# Depois é só adicionar o .gitignore e commitar a remoção
git commit -m "Removendo pasta enviada por engano"
```

4. "Professor, escrevi a mensagem do commit errado (ou com erro de português)!"

O que aconteceu: Na pressa, o aluno digitou `git commit -m "fiz a litsa"` e quer arrumar antes de enviar para o GitHub para não passar vergonha.

Como corrigir:

Desde que ele **ainda não tenha dado o `git push`**, ele pode reescrever o último commit facilmente:

```
git commit --amend -m "Resolvendo os exercícios da Lista 3"
```

Isso substitui a mensagem anterior pela nova.

5. "Apaguei um pedaço do código sem querer e salvei o arquivo!"

O que aconteceu: O aluno estava mexendo na estrutura HTML, deletou uma `div` importante, salvou o arquivo e se perdeu. Ele quer voltar o código para o jeito que estava na última vez que ele fez um commit.

Como corrigir:

O Git joga fora as alterações não "commitadas" e restaura o arquivo para a última versão salva com segurança:

```
git restore index.html
```

Dica de sala de aula: Quando um aluno cometer um erro muito grande e não souber explicar o que fez, o comando `git status` é o seu melhor amigo. Peça para ele rodar isso antes de tentar qualquer correção. Ele vai te mostrar exatamente quais arquivos estão travando o processo e o que o Git está esperando que o aluno faça.